Informe post-incidente — TP Unidad 6: respaldo y recuperación de Food Store

Bases de Datos II · Grupo 11 · Emmanuel Miranda · Simulacro: 28/10/2026 · Entorno: Windows + Git Bash, PostgreSQL 18.6 local. `foodstore` es una copia de la base de trabajo `food_store` (`createdb -T`), así que la original no se tocó: 6 categorías, 6 usuarios, 17 productos, 10 pedidos y 22 detalles.

Dato	Valor
Respaldo utilizado	`foodstore_20261028_225715.dump`, tomado el **28/10/2026 22:57:15 (-03)** · formato custom (`-Fc`) · 32 kB · 72 objetos en el TOC
Tiempo de restauración y validación	`createdb` 22:58:39 → validaciones OK 22:58:41: **1,3 s** (de esos, `pg_restore` 0,7 s)

RPO y RTO. El RPO mide **cuántos datos se pierden**: lo escrito desde el último respaldo hasta la falla. El RTO mide **cuánto se tarda en volver a operar**.

	Previsto (SPEC_U6)	Observado
RPO	24 h (respaldo diario)	**84 s** de antigüedad del respaldo al empezar el simulacro. En el laboratorio no hubo escrituras en esa ventana, así que la pérdida efectiva fue 0. En producción se perdería todo lo vendido desde el último respaldo, hasta 24 h.
RTO	15 min	**1,3 s**, muy por debajo del objetivo por el volumen chico. Un ensayo previo con el seed masivo (200k pedidos, 146 MB) dio 3,0 s.

Validaciones sobre `foodstore_restore_test`. Conteos idénticos a los registrados antes del respaldo (`.counts`): categoria 6 · usuario 6 · producto 17 · pedido 10 · detalle_pedido 22. `productos_invalidos = 0` y `detalles_invalidos = 0`. Controles extra: todas las foreign keys restauradas y validadas, y restricciones y triggers presentes.

Problema real encontrado. El primer intento del simulacro (22:57:41) terminó en `FALLIDO`, aunque el restore, los conteos y los inválidos estaban bien. Un control extra de FKs huérfanas asumía que la clave de `pedido` se llamaba `id_pedido`, y en nuestra base se llama distinto. Se reemplazó por la verificación de `pg_constraint.convalidated`, que no depende de nombres de columna, y la segunda corrida (22:58:39) dio todo OK. Al pasar los scripts del laboratorio a Windows aparecieron otros tres problemas: (1) la validación del PDF cuenta la tabla `cliente`, que **no existe** en nuestro schema (es `usuario`); (2) la base de clase no tiene la vista materializada que `health_check` y `maintenance` consultaban, y ahora se detecta con `to_regclass` y se omite; (3) `psql` en Windows termina las líneas con `\r\n` y rompía el log y la comparación de conteos, ahora se filtra con `tr -d '\r'`. Como la copia hecha con `createdb -T` arrancó **sin estadísticas**, el `ANALYZE` de `maintenance.sql` quedó justificado por el diagnóstico previo. `pedido` mostró 330 % de tuplas muertas (33 contra 10 vivas), pero en una sola página de 8 kB: no justifica un VACUUM, y por eso el script exige además más de 1000 muertas.

Mejora concreta. Programar `restore_test.sh` semanal en el Programador de tareas de Windows, en vez de correrlo sólo para el TP. El primer intento demostró que un procedimiento que no se ensaya en el entorno real falla justo cuando se lo necesita. Ensayarlo cada semana detecta estas diferencias antes de un incidente. Para el tiempo, `pg_restore --jobs=N` (ya soportado con `JOBS=4`) bajó la mediana de 3,3 a 2,7 s en el ensayo masivo. Para bajar el RPO por debajo de 24 h hace falta archivado de WAL (PITR), no más dumps.

Trabajo asistido con IA. Las primeras versiones de `SPEC_U6.md`, los scripts `.sh`, `health_check.sql` y `maintenance.sql` se generaron con un asistente de IA a partir de la consigna, `SPEC_U6.md` y `AGENTS.md`. Ningún script se aceptó sin correrlo en dos entornos: un laboratorio Linux con PostgreSQL 16 y mi equipo Windows con PostgreSQL 18. Las pruebas detectaron estos errores de la IA: el `ANALYZE` de la MV estaba antes del `REFRESH`; la prueba de falla del backup no ejercía `pg_dump`; usaba `bc`, que Git Bash no trae; daba por hecho que existían la MV y la columna `id_pedido`; y no contemplaba el `\r\n` de Windows. Ningún script contiene contraseñas: se usa `pgpass.conf` con `--no-password`. Evidencia completa en `evidencias/01` a `04`.